# Cloud agent environments

Copy

Environments give cloud agents a repeatable container, repositories, and setup for every cloud agent run.

Environments describe *how* an agent executes a task, not *what* it does. They give cloud agents the same container, repositories, and setup every time they run. Use an environment for a cloud agent run that needs a repeatable toolchain. Interactive local runs use your current checkout and machine setup, so they don't need one.

## What an environment includes

An environment groups the runtime configuration for a cloud agent run:

- **Docker image** - The image that provides the toolchain and dependencies for your code. A self-hosted Kubernetes worker with a `default_image` can run without a separate environment.
- **Repositories** - One or more repos that the agent clones into its workspace.
- **Setup commands** - Commands that prepare the workspace, such as dependency installation, builds, or code generation.
- **Environment variables** - Runtime values that you set in the Docker image or container configuration.
- **Agent Secrets** - Credentials and sensitive values that Warp injects at runtime. Configure them separately with Agent Secrets.

Together, these settings create a fresh workspace for each run. Warp provides prebuilt dev images with common languages and tools. You can also use an official image or publish your own.

## How environments fit into cloud agent runs

When the Automation Platform starts a cloud agent run, it combines the environment with a host, an agent profile, and task-specific context. Each part serves a distinct purpose:

- **Host** - Determines where the run executes. Choose Warp-hosted infrastructure or self-hosted runners.
- **Agent Profiles** - Set the agent's permissions, model choice, and defaults. See Agent Profiles.
- **Rules** - Provide instructions that guide agent responses and decisions. See Rules.
- **MCP servers** - Connect agents to external tools and data. See MCP servers.

- **Per-run context** - Supplies task-specific data, such as a Slack thread, PR metadata, or CI logs.

## When to use an environment

Use an environment when your run needs a predictable toolchain and repeatable setup. This is common in the following cases:

- **Integrations and schedules** - Runs from Slack, Linear, GitHub Actions, or a schedule need the same workspace each time.
- **CI and remote automation** - An environment prevents different runners or base images from changing the result.
- **Team workflows** - A shared environment gives every teammate the same image, repos, and setup commands.
- **Toolchain-specific work** - Use an environment when the workflow depends on particular language versions, linters, build tools, or system packages.

You can skip an environment for an interactive local run in a working checkout. The local agent uses your existing machine setup.

## Container users and permissions

Cloud agents run as a non-root user inside the container. See configuring container users for image, setup-command, and migration requirements.

## Related pages

- Configuring cloud agent environments to create, configure, and manage environments.
- Troubleshooting cloud agent environments to fix setup, authorization, permissions, and image failures.
- Runners to configure the compute that hosts environments.
- Deployment patterns to choose between Warp-hosted and self-hosted execution.

---